MOHAMED BIN ZAYED UNIVERSITY OF ARTIFICIAL INTELLIGENCE

Mohamed bin Zayed
University of
Artificial Intelligence

SWM Engineering

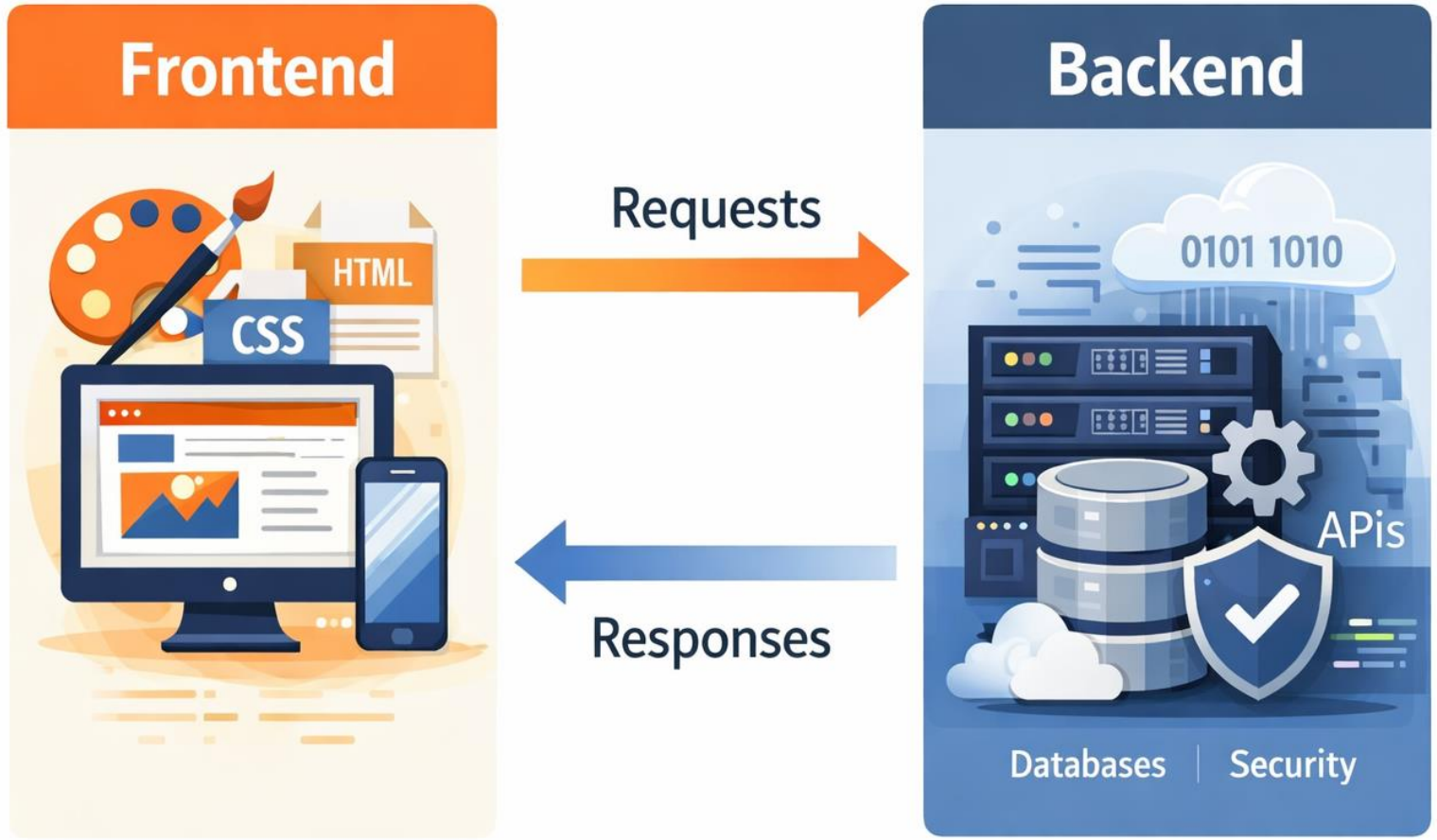
Date
4/2/2026

4.1

Frontend vs Backend

Why Two ENDS?

Frontend handles user interaction, while **backend** handles data processing, logic, and security; separating them **improves maintainability, scalability, and security**.



Is there a Clear Cut?

Short answer: No

However, there are some commonly accepted responsibilities that usually form a practical dividing line.

- If users can tamper with it and gain advantage, it belongs on the backend.
- If it exists to improve experience or presentation, it belongs on the frontend.

Why Two ENDS?

Aspect	Frontend	Backend
Runs on	User’s browser (client side)	Server
Focus	User interface and interaction	Logic, data, and processing
What users see	Yes	No
Handles UI	Yes	No
Handles databases	No	Yes
Security	Limited	Main responsibility
Languages	HTML, CSS, JavaScript	C, Java, Python, Node.js, etc.
Examples	Buttons, forms, pages	APIs, authentication, databases
Main goal	Good user experience	Correct and secure functionality

Frontend: the basics

The frontend is the part of a website or app users directly interact with. It includes the **layout**, **design**, and **interactive elements**, creating a smooth and **user-friendly experience**. It's called the client-side because it runs in the user's browser.

- **User Interface (UI) Design**: Frontend development focuses on designing the look of a website, including layout, colors, typography, and interactive elements.
- **Responsiveness**: Ensures that websites work well on various devices and screen sizes, providing a seamless experience on desktops, tablets, and smartphones.
- **Interactivity**: It implements interactive features like buttons, forms, animations, and dynamic content that engage users.
- **Cross-Browser Compatibility**: Ensures that the website functions correctly across different web browsers, such as Chrome, Firefox, and Safari.

Frontend: the basics

Frontend is the **user-facing part** of an application

- What users **see**
- What users **click**
- What users **interact with**

It runs on the **client side** (usually a web browser, or any other app)

Concept	Meaning
UI (User Interface)	Visual layout
UX (User Experience)	How easy it is to use
Responsive design	Works on phone & desktop
Events	Click, input, submit
DOM	Browser’s page structure

Frontend: development tools (core and build)

Category	Common Tools
Core languages	HTML, CSS, JavaScript, TypeScript
UI frameworks	React, Vue, Angular, Svelte
Meta-frameworks (SSR)	Next.js, Nuxt, Remix, Astro
Styling tools	CSS, Sass, PostCSS, Tailwind CSS
UI component libraries	MUI, Ant Design, Bootstrap, shadcn/ui
Build tools & bundlers	Vite, Webpack, Rollup, esbuild
Package managers	npm, yarn, pnpm

Frontend: development tools (data, quality and delivery)

Category	Common Tools
State management	Redux Toolkit, Zustand, MobX
Data fetching	Fetch API, Axios, React Query, SWR
Routing	React Router, Next.js Router, Vue Router
Testing	Jest, Vitest, Testing Library, Cypress
Code quality	ESLint, Prettier, TypeScript
Debugging	Browser DevTools, React DevTools
Performance	Lighthouse, Web Vitals
Accessibility	axe DevTools, Lighthouse a11y
Design & collaboration	Figma, Storybook
Deployment & hosting	Vercel, Netlify, Cloudflare Pages

Frontend: example tools

- **HTML** (Hypertext Markup Language): The standard language for creating the structure of web pages. It defines elements such as headings, paragraphs, links, and images.
- **CSS** (Cascading Style Sheets): Used for styling HTML elements. CSS controls the layout, colors, fonts, and responsiveness of a webpage.
- **JavaScript**: A programming language that adds interactivity to the web. JavaScript is used to create dynamic content, such as forms, animations, and interactive maps.
- **Frontend Frameworks**: React, Angular, Tailwind, Bootstrap, and Vue.js.
- **Frontend Libraries**: jQuery and SASS.

Frontend: HTML

HTML (HyperText Markup Language) is the standard **markup language** used to create and structure content on the web. HTML is **not a programming language** (it has no logic).

- Defines the **structure** of a web page
- Organizes content using **elements (tags)**
- Tells the browser *what* each piece of content is

```
<!DOCTYPE html>
<html>
 <head>
  <title>My First Web Page</title>
 </head>
 <body>
  <h1>Hello World!</h1>
  <p>This is a paragraph.</p>
 </body>
</html>
```

Frontend: CSS

CSS (Cascading Style Sheets) is the language used to **style and design** HTML content.
It controls **how things look**, while HTML controls **what things are**.

- Change **colors** and **fonts**
- Control **layout** and **spacing**
- Make pages **responsive** (work on phones & desktops)
- Add simple **animations**

Frontend: CSS

HTML + CSS **together** create a **complete web page**:

- HTML = content & structure
- CSS = style & presentation

```
<p class="message">Hello World</p>

.message {
  color: blue;
  font-size: 20px;
  font-weight: bold;
}
```

- Text is **blue**
- Text is larger
- Text is **bold**

Frontend: CSS

demo.html

```html
<!DOCTYPE html>
<html>
  <head>
   <title>My Simple Web Page</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
   <div class="card">
    <h1>Welcome!</h1>
    <p>HTML and CSS working together.</p>
    <button>Click Me</button>
   </div>
  </body>
</html>
```

styles.css

```css
body {
 font-family: Arial, sans-serif;
 background-color: #f2f2f2;
}

.card {
 background: white;
 width: 300px;
 padding: 20px;
 margin: 40px auto;
 border-radius: 8px;
 text-align: center;
}

button {
 background-color: #007bff;
 color: white;
 border: none;
 padding: 10px 16px;
 border-radius: 4px;
}
```

Frontend: JavaScript

JavaScript (JS) is a programming language used to make web pages **interactive and dynamic**. While HTML structures content and CSS styles it, **JavaScript controls behavior**.

- Respond to **user actions** (clicks, typing)
- Change **HTML content** and **CSS styles**
- Show alerts, popups, and messages
- Validate forms
- Build interactive features (menus, sliders, games)

Frontend: HTML + CSS + JavaScript

demo.html

```html
<!DOCTYPE html>
<html>
  <head>
   <title>My Simple Web Page</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
   <div class="card">
    <h1>Welcome!</h1>
    <p>HTML and CSS working together.</p>
    <button onclick="changeText()">Click Me</button>
   </div>
   <script src="script.js"></script>
  </body>
</html>
```

script.js

```javascript
function changeText() {
  document.getElementById("message").innerText =
"You clicked the button!";
}
```

styles.css

```css
body {
 font-family: Arial, sans-serif;
 background-color: #f2f2f2;
}

.card {
 background: white;
 width: 300px;
 padding: 20px;
 margin: 40px auto;
 border-radius: 8px;
 text-align: center;
}

button {
 background-color: #007bff;
 color: white;
 border: none;
 padding: 10px 16px;
 border-radius: 4px;
}
```

Frontend: TypeScript

TypeScript (TS) is a programming language built **on top of JavaScript**. It adds **types** to JavaScript to make code **safer, clearer, and easier to maintain**, especially in large projects.

JavaScript is very flexible, but that flexibility can cause **bugs**. TypeScript helps by:

- Catching errors **before the code runs**
- Making code easier to **read and understand**
- Helping editors give **better autocomplete and hints**

JavaScript

```
function add(a, b) {
  return a + b;
}

add(5, "3"); // works, but may cause bugs
```

TypeScript

```
function add(a: number, b: number): number {
  return a + b;
}

add(5, 3);      // OK
add(5, "3");    // Type error
```

Frontend: UI Frameworks

True thing: with HTML + CSS + JavaScript/TypeScript we can build anything.
The issue is how flexible and maintainable the frontend is, and how do we combat **scale** and **complexity**. They describe **what** the browser can do, not **how to structure large applications**.

Without a framework, you must design and maintain:

- State management
- DOM (Document Object Model) update logic
- Event orchestration
- Component boundaries
- Data - UI synchronization
- Performance optimizations (diffing, batching, memoization)

Frameworks provide **abstractions** over these recurring problems.

UI Frameworks

Framework	Style	Learning Curve	Typical Use
React	Flexible	Medium	Industry default
Angular	Opinionated	High	Enterprise apps
Vue	Progressive	Low–Medium	General-purpose

Flexible: The framework **imposes few rules** and lets developers choose **how to structure, architect, and extend** the application.

Opinionated: The framework **makes many decisions for you** about *how* your application should be structured, written, and organized.

Progressive: You can **adopt the framework gradually**, using as much or as little of it as you need, **without rewriting your entire application**.

UI Frameworks: React

React is a JavaScript library (maintained by Meta) for building user interfaces using **declarative, component-based rendering**, where the UI automatically updates in response to state changes.

Features:

- **Declarative UI**: You describe *what* the UI should look like for a given state, and React handles updating the DOM.
- **Component-based**: UIs are built from reusable components that manage their own logic and state.
- **State-driven rendering**: When state or props change, React re-renders the affected components.
- **Virtual DOM & diffing**: React compares previous and new UI representations and applies only the necessary DOM updates.
- **Unidirectional data flow**: Data flows from parent to child, making behavior predictable.
- **Ecosystem-first**: React focuses on UI; routing, data fetching, and state management are handled via libraries.
- **Platform-agnostic**: The same mental model works for web (React), mobile (React Native), and more.

UI Frameworks: React

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>HTML + React Example</title>
</head>
<body>
  <!-- Plain HTML -->
  <h1>Static HTML Header</h1>
  <p>This paragraph is pure HTML.</p>

  <!-- React mount point -->
  <div id="react-root"></div>

  <!-- More plain HTML -->
  <footer>© 2026</footer>

  <!-- React -->
  <script
src="https://unpkg.com/react@18/umd/react.development.js"></s
cript>
  <script src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js"></script>
  <script
src="https://unpkg.com/@babel/standalone/babel.min.js"></scri
pt>
```

```html
<!-- React code -->
  <script type="text/babel">
    function Greeting() {
      const [name, setName] = React.useState("World");

      return (
        <div>
          <h2>Hello, {name}!</h2>
          <input
            value={name}
            onChange={e => setName(e.target.value)}
          />
        </div>
      );
    }

    const root = ReactDOM.createRoot(
      document.getElementById("react-root")
    );

    root.render(<Greeting />);
  </script>

</body>
</html>
```

UI Frameworks: Angular

Angular is a **full, opinionated frontend framework** (maintained by Google) for building **large, structured web applications**.

Features:

- Angular includes routing, forms, HTTP client, dependency injection, testing tools, and build setup out of the box.
- It enforces specific patterns (modules, components, services), reducing architectural ambiguity across teams.
- Angular is built around TypeScript for static typing, tooling, and large-scale maintainability.
- Applications are composed of components with templates, logic, and styles.
- Templates can automatically sync UI - state, useful for forms.
- Services are injected rather than manually instantiated, improving testability and structure.

UI Frameworks: Vue

Vue is a **progressive JavaScript framework** for building user interfaces and single-page applications (SPA) or multi-page applications (MPA).

Features:

- You can drop Vue into an existing HTML page or scale it up to a full app.
- Vue tracks dependencies and updates only what changed.
- UI is built from reusable components.
- `*.vue` files combine template, logic, and styles in one place.
- HTML-like templates + simple reactivity model.

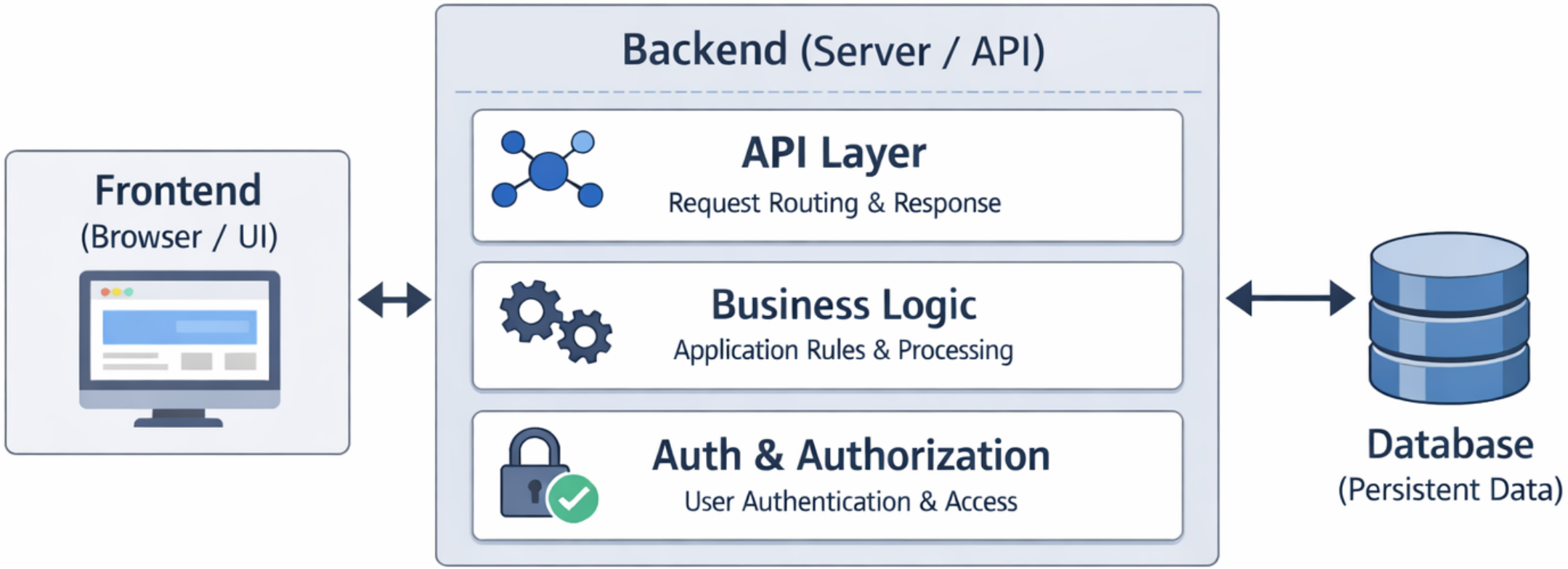
Backend: the basics

The **backend** is the part of a system that **runs on the server** and is responsible for **data, business logic, and system coordination**, not the user interface.

A backend typically handles:

- Rules like validation, permissions, pricing, workflows.
- Reading/writing databases (SQL, NoSQL).
- Exposes endpoints (HTTP, GraphQL, gRPC) that frontends call.
- Login, sessions, tokens, access control.
- Input validation, rate limiting, encryption.
- Communicates with external services (payments, email, analytics).

Backend: components



Backend: components

Server

A long-running program that:

- Listens for requests (HTTP, WebSocket)
- Executes business logic
- Returns responses

API layer

Defines how clients communicate with the backend.

Common styles:

- REST
- GraphQL
- gRPC

Business logic

The “rules of the system”.

Examples:

- You can’t transfer money you don’t have
- Discounts apply only to certain users
- Orders must follow a workflow

Authentication & authorization

Determines:

- Who the user is (auth)
- What they can do (authorization)

Examples:

- Passwords
- OAuth
- JWT
- Sessions

Database

Persistent storage:

- User accounts
- Orders
- Messages
- Logs

Backends abstract database access so:

- Data remains consistent
- Constraints are enforced

Example Application: playing chess

What is the app?

An **online chess platform** where users can:

- Create an account
- Start or join a game
- Make moves
- See opponent moves
- Win, lose, or draw

Goal:

- Show why frontend and backend must be separated

Example Application: playing chess

Functional Requirements

- Sign up and log in
- Start a new chess game
- Play against another user
- Make legal moves only
- See real-time board updates
- View game result (win / loss / draw)

Example Application: playing chess

Non-Functional Requirements

- Game rules must be enforced correctly
- Players cannot cheat by modifying the UI
- Moves must be synchronized in real time
- Game state must persist
- Low latency communication

Example Application: playing chess

Frontend Responsibilities

- Rendering the chessboard
- Animations and highlights
- Capturing user clicks
- Sending moves to backend
- Displaying opponent moves
- Showing timers and status

Technologies (example)

- React / Vue
- HTML, CSS, JavaScript

Example Application: playing chess

Backend Responsibilities

- Player authentication
- Game creation & matchmaking
- Validating chess moves
- Enforcing turn order
- Detecting check, checkmate, draw
- Broadcasting updates to players

Example Application: playing chess

Database Responsibilities

- Users
- Games
- Moves history
- Current board state

Tables in the database (examples)

```
users(uid, username)
games(gid, status, current_turn)
moves(mid, gid, from, to)
...
```

Example Application: playing chess

Frontend

User needs
- Sign-up / login
- Start a new game
- Send moves to backend

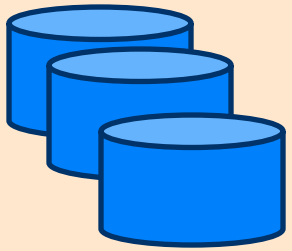
Technologies
- React/Vue
- HTML,CSS, JS

Backend (Game Server)

API Layer
- HTTP/WebSockets

Game Logic
- Player authentication
- Validating moves
- Enforcing rules
- Broadcasting updates

Database System



Storage
- Users
- Games
- Moves history
- Statistics

Other Systems

Mohamed bin Zayed
University of
Artificial Intelligence

Thank you